

TASK-EFFICIENT AGENT EVALUATION USING CACHED TRACES

Anonymous authors

Paper under double-blind review

ABSTRACT

Efficient benchmarking estimates a system’s full-benchmark performance from a small number of task executions. Standard approaches rely solely on the binary correctness of executions from a collection of already-evaluated reference systems. An agentic system, however, produces a rich trace for each execution that can reveal substantially more about its capabilities. Exploiting this information requires useful trace representations. We propose representing a trace by extracting a structured, rubric-guided summary of it with an LLM and embedding the summary with a standard text encoder. These representations preserve correctness- and task-related signal while minimizing the presence of system identity-, model family-, and harness-related signals that dominate raw trace embeddings. Given this representation for each task execution from all reference systems, we construct low-dimensional representations of the systems, predict a new system’s score from its position among the reference systems, and combine the resulting geometric prediction with a two-parameter logistic item-response model. On SWE-bench Verified and Terminal-Bench 2.0, the proposed estimator reduces mean absolute error from 0.13 to 0.08 ($p < 10^{-4}$) and from 0.14 to 0.12 ($p < 10^{-3}$), respectively, when provided a single random executed task relative to an item-response model alone. These results indicate that trace geometry and correctness outcomes carry complementary predictive signal and enable more accurate benchmark score estimation from limited agent executions.

1 INTRODUCTION

The dominant paradigm of generative model use is increasingly agentic: systems plan, invoke tools, observe intermediate results, and iterate over long horizons to complete complex tasks (Yang et al., 2024; Wang et al., 2025). As agentic capabilities increase, so does the range of settings in which a given system might be deployed – and, with it, the number of benchmarks on which the system must be evaluated before it can be trusted with consequential work (Kapoor et al., 2026). Agentic evaluation, however, is expensive: each task execution requires a properly sandboxed environment and a single execution routinely consumes hundreds of thousands of tokens (Jimenez et al., 2024; Terminal-Bench Team, 2026). At reported per-task costs of \$0.13–\$4 (Kapoor et al., 2026), the cost of evaluating a single system on a single benchmark can regularly exceed \$2,000. Given this cost, efficient evaluation is necessary to increase the evaluation coverage of a system.

Task-efficient evaluation methods – via benchmark compression and anchor-item selection (Vivek et al., 2024; Maia Polo et al., 2024), item response theory and adaptive testing (Kipnis et al., 2025; Truong et al., 2025), leveraging cached responses (Helm et al., 2026a), etc. – address this problem by finding structure in item-level information available from previously-evaluated reference systems. Some task-efficient evaluation methods require only the item-level scores of each reference system (Vivek et al., 2024; Kipnis et al., 2025). Others leverage the content of cached artifacts, such as the natural language responses or residual streams, to measure the similarity between systems beyond its score on a set of tasks (Duderstadt et al., 2023; Helm et al., 2026b;a). Notably, agentic workflows often produce artifacts when executing a task – file diffs, tool invocations, intermediate plans, and error messages – recorded together in an execution trace that could be used to this effect. Unfortunately, existing methods for using this non-score information rely on off-the-shelf embedding functions that are not immediately well-suited to embedding traces.

We make two primary contributions:

Useful trace representations. We introduce TRUBRIC (*trace rubric*): a trace representation constructed by extracting a structured, rubric-guided summary of each trace with an LLM and embedding the summary with a standard text encoder. We show that TRUBRIC embeddings preserve information relevant to the task and behavior of the system while mitigating the effect of the specific system, model family, or harness used to produce the trace across seven different modern text encoders.

Trace-efficient evaluation. Given the TRUBRIC representations of traces from a set of tasks for a target system and a collection of reference systems, we evaluate their utility for predicting the target system’s full-benchmark score on SWE-bench Verified and Terminal-Bench 2.0 under three different agentic system evaluation protocols. We find that TRUBRIC-based estimators, blended with an item-response model, reduce single-task mean absolute error from 0.13 to 0.08 ($p < 10^{-4}$) and from 0.14 to 0.12 ($p < 10^{-3}$) on the two benchmarks respectively, and are never less accurate than their score-only counterparts at any task budget. Ablations show that the gains grow with the number of reference systems, compose with adaptive task selection, and are insensitive to the choice of text encoder.

1.1 PROBLEM STATEMENT

Given an agentic system f , we consider the problem of predicting its score on a benchmark $Q^* = \{q_1, \dots, q_M\}$. For our purposes, a system $f \in \mathcal{F}$ is a random mapping from a query space $\mathcal{Q}$ to a trace space $\mathcal{X}$: executing f on query q yields a trace $f(q)$, a long, semi-structured log of the system’s actions, observations, and outputs. Let $s : \mathcal{X} \rightarrow \{0, 1\}$ denote the harness’s trace-level scoring function and $y : \mathcal{F} \times 2^{Q^*} \rightarrow [0, 1]$ denote the benchmark scoring function, $y(f, Q) = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{E}[s(f(q))]$. We refer to $y(f, Q^*)$ as y when the context is clear.

As described above, full evaluation of f on all queries may be infeasible. Instead, we assume access to n previously-evaluated reference systems $f_1, \dots, f_n$ for which we observe both the execution artifacts and the item-level scores on every query:

$$\mathcal{D} = \left\{ \{(x_{ij}, y_{ij})\}_{j=1}^M \right\}_{i=1}^n,$$

where $x_{ij} \sim f_i(q_j)$ is the trace of reference system f_i on query q_j and $y_{ij} = s(x_{ij}) \in \{0, 1\}$ is its trace-level score. The benchmark score of reference system f_i is $y_i = \frac{1}{M} \sum_{j=1}^M y_{ij}$. Such reference caches exist publicly at scale.¹ Given $\mathcal{D}$, our goal is to estimate y with $m \ll M$ queries. That is, we seek an estimate $\hat{y}$ that predicts the full benchmark score of f from its traces (and, optionally, its trace-level scores) on a subset $Q \subset Q^*$ with $|Q| = m$, together with $\mathcal{D}$.

Two aspects distinguish this setting from the short-response setting of Helm et al. (2026a). First, the observation associated with each query is not a short response but a trace of 10^4 – 10^6 tokens in one of many mutually incompatible harness formats. Whether and how traces can be embedded usefully is the central question of this paper. Second, the reference population is confounded: reference systems that share the target’s underlying language model or harness permit trivially accurate prediction via score-copying. We therefore evaluate under a leave-one-model-family-out protocol – references sharing the target’s underlying model family are excluded – and verify robustness under a leave-one-harness-out protocol. The query subset Q may be fixed, drawn at random, or selected adaptively per target; we report all three regimes.

¹The SWE-bench Verified leaderboard archive contains traces and outcomes for over one hundred systems; we estimate it embodies \$50K–\$100K of agent compute contributed by submitters.

2 METHOD

3 TRACE REPRESENTATIONS ENCODE CONFOUNDERS

4 EFFICIENT EVALUATION

5 RELATED WORK

6 DISCUSSION

REFERENCES

- Brandon Duderstadt, Hayden S. Helm, and Carey E. Priebe. Comparing foundation models using data kernels. *arXiv preprint arXiv:2305.05126*, 2023.
- Hayden Helm, Johnson, and Carey Priebe. Query-efficient model evaluation using cached responses. In *International Conference on Machine Learning (ICML)*, 2026a. arXiv:2605.07096.
- Hayden Helm, Xiaodong Liu, and Weiwei Yang. Jailbreak susceptibility prediction and mitigation via the behavioral geometry of models. *arXiv preprint arXiv:2605.26409*, 2026b.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- Sayash Kapoor, Benedikt Stroebel, et al. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.11977.
- Alex Kipnis et al. metabench: A sparse benchmark to measure general ability in large language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.12844.
- Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.
- Terminal-Bench Team. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- Sang Truong et al. Reliable and efficient amortized model-based evaluation. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.13335.
- Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024. arXiv:2309.08638.
- Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.16741.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.